

BAB 6: Authorization, Middleware & Role-Based Access Control (RBAC)

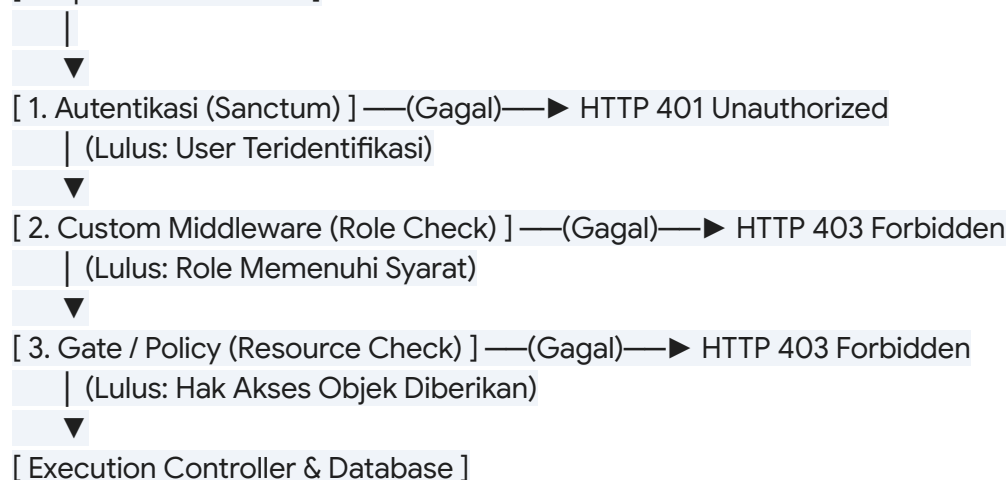
BAGIAN 1: RINGKASAN MATERI PEMBELAJARAN MENDALAM (DEEP LEARNING)

1. Mendasari Perbedaan: Authentication vs Authorization

Sebelum membatasi sistem, seorang Backend Developer harus membedakan dengan jelas dua pilar keamanan dasar:

- **Authentication (Autentikasi):** Proses membuktikan **siapa Anda** (Identity Check). Contoh: Login membawa *email & password*, kemudian server menerbitkan *Bearer Token*.
- **Authorization (Otorisasi):** Proses menentukan **apa yang boleh Anda lakukan** (Access Permission Check). Contoh: Meskipun *Siswa* sudah login, dia tetap dilarang menghapus data *Guru* atau meng-upgrade role pengguna lain.

[Request HTTP Masuk]



2. Mengenal Custom Middleware (HTTP Filtering)

Middleware bertindak sebagai **pintu penyaring (filter)** yang berada di antara permintaan HTTP klien dan logika aplikasi di Controller.

- **Mekanisme Kerja:** Middleware menerima objek `$request` dan penangan `$next`.
- **Proses Evaluasi:** Jika kondisi peran pengguna terpenuhi, permintaan diteruskan ke `$next($request)`. Jika gagal, Middleware langsung menghentikan eksekusi dan mengembalikan respons **HTTP 403 Forbidden**.

3. Konsep RBAC (Role-Based Access Control)

RBAC adalah pendekatan mengelompokkan hak akses berdasarkan **Peran (Role)** dalam organisasi/sistem:

Role	Hak Akses Utama (Scope Permission)	Contoh Aksi
Admin	Akses Penuh (<i>Full Privileges</i>) ke seluruh <i>resource</i> .	Menambah/menghapus akun Guru & Siswa, mengelola master data.
Guru / Pembimbing	Akses Pengelolaan Konten Akademik & Penilaian.	Mengisi nilai, mengunggah materi, melihat daftar siswa.
Siswa / User	Akses Baca (<i>Read-Only</i>) & Terbatas pada Resource Milik Sendiri.	Melihat nilai pribadi, mengunduh materi, mengubah profil sendiri.

4. Fine-Grained Authorization: Gate vs Policy

Laravel menyediakan dua fitur canggih untuk menangani otorisasi secara presisi:

A. Laravel Gate (Otorisasi Global/Sederhana)

- Berorientasi pada tindakan global tanpa tergantung pada objek *Model* tertentu.
- Sangat cocok untuk pemeriksaan role cepat atau aksi umum.
- Contoh: `Gate::allows('akses-dashboard-admin')`.

B. Laravel Policy (Otorisasi Berbasis Resource/Model)

- Berorientasi pada **Model** data tertentu (misalnya `GradePolicy` untuk model `Grade`).
- Menentukan hak akses spesifik terhadap satu baris data (*instance level*).
- Contoh: Seorang Guru hanya boleh mengubah nilai jika mata pelajaran tersebut benar-benar diampu olehnya.

BAGIAN 2: JOBSHEET PRAKTEK

INTENSIF (PROJECT HANDBOOK)

- **Judul Proyek:** *Restricted API Middleware & Role Permission Management* (toko-rbac-rpl)
- **Waktu Pengerjaan:** 1.620 Menit (36 JP)
- **Target Output:** API Backend Laravel dengan 3 Role (*Admin, Guru, Siswa*), Custom Middleware CheckRole, Policy Pengamanan Resource, serta Frontend Pengetesan menggunakan Vanilla JS (Fetch API).

TAHAP 1: PERSIAPAN BACKEND LARAVEL

Step 1: Skema Database & Migration Modifikasi User

Buka berkas migrasi database/migrations/xxxx_xx_xx_XXXXXX_create_users_table.php dan tambahkan kolom role:

```
public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        // Menambahkan ENUM Role sesuai spesifikasi tugas
        $table->enum('role', ['admin', 'guru', 'siswa'])->default('siswa');
        $table->rememberToken();
        $table->timestamps();
    });
}
```

Ubah berkas app/Models/User.php agar mendukung *fillable* role dan trait Sanctum:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
```

```
protected $fillable = [
    'name',
    'email',
    'password',
    'role',
];

protected $hidden = [
    'password',
    'remember_token',
];
}
```

Jalankan ulang migrasi dan buat seeder awal:

Bash

```
php artisan migrate:fresh
```

Step 2: Membuat Database Seeder untuk Testing Role

Buat Seeder untuk memposting data akun pengujian 3 role:

Bash

```
php artisan make:seeder UserRoleSeeder
```

Isi berkas database/seeder/UserRoleSeeder.php:

PHP

```
namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class UserRoleSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name' => 'Administrator Utama',
            'email' => 'admin@sekolah.sch.id',
            'password' => Hash::make('password123'),
            'role' => 'admin',
        ]);
    }
}
```

```
]);
```

```
User::create([
    'name' => 'Pak Budi (Guru RPL)',
    'email' => 'guru@sekolah.sch.id',
    'password' => Hash::make('password123'),
    'role' => 'guru',
]);
```

```
User::create([
    'name' => 'Siswa Pembelajar',
    'email' => 'siswa@sekolah.sch.id',
    'password' => Hash::make('password123'),
    'role' => 'siswa',
]);
}
}
```

Jalankan seeder:

Bash

```
php artisan db:seed --class=UserRoleSeeder
```

Step 3: Membuat Custom Middleware CheckRole

Jalankan perintah berikut di terminal:

Bash

```
php artisan make:middleware CheckRole
```

Buka berkas app/Http/Middleware/CheckRole.php dan tuliskan logika verifikasinya:

PHP

```
namespace App\Http\Middleware;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
use Symfony\Component\HttpFoundation\Response;
```

```
class CheckRole
```

```
{
```

```
    /**
```

```
     * Memeriksa apakah user yang terautentikasi memiliki role yang sesuai.
```

```
     *
```

```
     * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
```

```
     * @param mixed ...$roles (menerima multiple role dipisah koma)
```

```

*/
public function handle(Request $request, Closure $next, ...$roles): Response
{
    // 1. Cek Apakah User Sudah Login
    if (!$request->user()) {
        return response()->json([
            'status' => false,
            'message' => 'Unauthenticated. Silakan login terlebih dahulu.'
        ], 401);
    }

    // 2. Cek Apakah Role User Terdaftar pada Array Permintaan
    if (in_array($request->user()->role, $roles)) {
        return response()->json([
            'status' => false,
            'message' => 'Forbidden Access. Role Anda ('.$request->user()->role.') tidak memiliki wewenang pada rute ini.'
        ], 403);
    }

    return $next($request);
}
}

```

Mendaftarkan Middleware ke Aplikasi:

- Jika menggunakan Laravel 11/12: Buka bootstrap/app.php dan tambahkan alias:

PHP

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\CheckRole::class,
    ]);
})

```

Step 4: Membuat Migration & Model Nilai/Tugas (Resource)

Kita buat satu model contoh pengelolaan data nilai siswa:

Bash

```
php artisan make:model Grade -m
```

Buka migrasi database/migrations/xxxx_xx_xx_XXXXXX_create_grades_table.php:

PHP

```
public function up(): void
{
    Schema::create('grades', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade'); // ID Siswa
        $table->string('subject'); // Mata Pelajaran
        $table->integer('score'); // Nilai
        $table->timestamps();
    });
}
```

Buka model app/Models/Grade.php:

PHP

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Grade extends Model
{
    use HasFactory;

    protected $fillable = ['user_id', 'subject', 'score'];

    public function student()
    {
        return $this->belongsTo(User::class, 'user_id');
    }
}
```

Jalankan migrasi:

Bash

```
php artisan migrate
```

Step 5: Membuat Policy GradePolicy

Buat Policy untuk membatasi aksi spesifik terhadap objek Nilai:

Bash

```
php artisan make:policy GradePolicy --model=Grade
```

Buka app/Policies/GradePolicy.php dan isi Aturan Otorisasi:

PHP

```
namespace App\Policies;

use App\Models\Grade;
use App\Models\User;

class GradePolicy
{
    /**
     * Admin dan Guru dapat melihat seluruh daftar nilai.
     * Siswa hanya bisa melihat jika nilai tersebut miliknya sendiri.
     */
    public function view(User $user, Grade $grade): bool
    {
        if ($user->role === 'admin' || $user->role === 'guru') {
            return true;
        }

        return $user->id === $grade->user_id;
    }

    /**
     * Hanya Admin dan Guru yang boleh menambah atau mengubah nilai.
     */
    public function create(User $user): bool
    {
        return in_array($user->role, ['admin', 'guru']);
    }

    /**
     * Hanya Admin yang boleh menghapus record nilai dari database.
     */
    public function delete(User $user, Grade $grade): bool
    {
        return $user->role === 'admin';
    }
}
```

Step 6: Controller & Penanganan Error Otorisasi

Buat controller API untuk nilai:

Bash

```
php artisan make:controller Api/GradeController
```

Isi berkas app/Http/Controllers/Api/GradeController.php:

PHP

```
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Grade;
use Illuminate\Http\Request;

class GradeController extends Controller
{
    // Menampilkan seluruh data nilai (Untuk Admin/Guru) atau milik siswa login
    public function index(Request $request)
    {
        $user = $request->user();

        if ($user->role === 'siswa') {
            $grades = Grade::where('user_id', $user->id)->with('student')->get();
        } else {
            $grades = Grade::with('student')->get();
        }

        return response()->json([
            'status' => true,
            'data' => $grades
        ]);
    }

    // Menambah Nilai Baru
    public function store(Request $request)
    {
        // Panggil Policy
        $this->authorize('create', Grade::class);

        $request->validate([
            'user_id' => 'required|exists:users,id',
            'subject' => 'required|string',
        ]);
    }
}
```

```

        'score' => 'required|integer|min:0|max:100'
    ]);

    $grade = Grade::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Nilai berhasil disimpan.',
        'data' => $grade
    ], 201);
}

// Menghapus Nilai
public function destroy(Request $request, $id)
{
    $grade = Grade::findOrFail($id);

    // Panggil Policy Delete (Hanya Admin)
    $this->authorize('delete', $grade);

    $grade->delete();

    return response()->json([
        'status' => true,
        'message' => 'Data nilai berhasil dihapus dari database.'
    ]);
}
}

```

Step 7: Menyiapkan Rute API Terproteksi (routes/api.php)

Atur routes/api.php agar menggabungkan **Auth Sanctum** dan **Custom Middleware CheckRole**:

PHP

```

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\GradeController;

// Rute Publik
Route::post('/login', [AuthController::class, 'login']);

```

// Rute Terproteksi Token Autentikasi

```
Route::middleware('auth:sanctum')->group(function () {
```

```
    Route::post('/logout', [AuthController::class, 'logout']);
```

```
    Route::get('/me', [AuthController::class, 'me']);
```

// 1. Rute yang Bisa Diakses Semua Role Login (Admin, Guru, Siswa)

```
    Route::get('/grades', [GradeController::class, 'index']);
```

// 2. Rute Khusus Admin & Guru (Menciptakan Nilai Baru)

```
    Route::middleware('role:admin,guru')->group(function () {
```

```
        Route::post('/grades', [GradeController::class, 'store']);
```

```
    });
```

// 3. Rute Eksklusif Khusus Admin (Menghapus Nilai / Manajemen User)

```
    Route::middleware('role:admin')->group(function () {
```

```
        Route::delete('/grades/{id}', [GradeController::class, 'destroy']);
```

```
    });
```

```
});
```

TAHAP 2: PROYEK FRONTEND INTEGRASI HAK AKSES (VANILLA JS)

Buat folder terpisah frontend-rbac-smk dengan 3 berkas dasar:

1. Berkas index.html

HTML

```
<!DOCTYPE html>
```

```
<html lang="id">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Sistem Akses Akademik RBAC - SMK RPL</title>
```

```
    <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
    <!-- KOTAK LOGIN -->
```

```
    <div id="kotak-login" class="kotak">
```

```
        <h2>🔒 Login Sistem RBAC</h2>
```

```
        <form id="form-login">
```

```
            <div class="form-grup">
```

```
                <label>Pilih Role Pengujian:</label>
```

```

        <select id="pilih-role">
            <option value="admin@sekolah.sch.id">Admin (admin@sekolah.sch.id)</option>
            <option value="guru@sekolah.sch.id">Guru (guru@sekolah.sch.id)</option>
            <option value="siswa@sekolah.sch.id">Siswa (siswa@sekolah.sch.id)</option>
        </select>
    </div>
    <div class="form-grup">
        <label>Password:</label>
        <input type="password" id="password" value="password123" required>
    </div>
    <button type="submit" class="btn biru">Login Ke Sistem</button>
</form>
<p id="pesan-error" class="error"></p>
</div>

<!-- KOTAK DASHBOARD -->
<div id="kotak-dashboard" class="kotak sembunyi">
    <div class="flex-header">
        <div>
            <h2><img alt="graduation cap icon" data-bbox="215 452 235 468"/> Dashboard Akademik</h2>
            <p>Login sebagai: <strong id="label-user">-</strong> | Role: <span id="badge-role"
class="badge">-</span></p>
        </div>
        <button id="btn-logout" class="btn merah">Logout</button>
    </div>
    <hr style="margin: 15px 0;">

    <!-- FORM TAMBAH NILAI (HANYA DITAMPILKAN UNTUK ADMIN/GURU) -->
    <div id="area-tambah-nilai" class="sembunyi" style="background: #eef6ff; padding: 15px; border-radius: 6px;
margin-bottom: 20px;">
        <h3><img alt="plus icon" data-bbox="200 655 220 675"/> Form Input Nilai (Khusus Admin & Guru)</h3>
        <form id="form-nilai" style="display: flex; gap: 10px; margin-top: 10px;">
            <input type="number" id="input-siswa-id" placeholder="ID Siswa (Contoh: 3)" required style="width:
120px;">
            <input type="text" id="input-mapel" placeholder="Mata Pelajaran" required>
            <input type="number" id="input-nilai" placeholder="Nilai (0-100)" required style="width: 100px;">
            <button type="submit" class="btn hijau">Simpan Nilai</button>
        </form>
    </div>

    <!-- TABEL DATA NILAI -->
    <h3><img alt="notepad icon" data-bbox="185 862 205 882"/> Daftar Nilai Akademik</h3>
    <table>

```

```

        <thead>
        <tr>
            <th>ID</th>
            <th>ID Siswa</th>
            <th>Mata Pelajaran</th>
            <th>Nilai</th>
            <th>Aksi (Khusus Admin)</th>
        </tr>
        </thead>
        <tbody id="tabel-data">
            <!-- Data Otomatis Diisi via JavaScript -->
        </tbody>
    </table>
</div>

<script src="app.js"></script>
</body>
</html>

```

2. Berkas style.css

CSS

```

body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background-color: #f4f6f9; padding: 20px; }

.kotak { background: white; max-width: 800px; margin: 20px auto; padding: 25px; border-radius: 8px;
box-shadow: 0 4px 10px rgba(0,0,0,0.08); }

.sembunyi { display: none !important; }

.form-grup { margin-bottom: 12px; }

.form-grup label { display: block; font-weight: bold; margin-bottom: 5px; }

.form-grup input, .form-grup select { width: 100%; padding: 8px; border: 1px solid #ccc; border-radius: 4px; }

.btn { padding: 8px 14px; border: none; border-radius: 4px; cursor: pointer; color: white; font-weight: bold; }

.biru { background-color: #007bff; }

.hijau { background-color: #28a745; }

.merah { background-color: #dc3545; }

.flex-header { display: flex; justify-content: space-between; align-items: center; }

.badge { background: #6c757d; color: white; padding: 3px 8px; border-radius: 12px; font-size: 12px;
text-transform: uppercase; }

.badge.admin { background: #dc3545; }

.badge.guru { background: #ffc107; color: black; }

.badge.siswa { background: #17a2b8; }

.error { color: #dc3545; font-weight: bold; margin-top: 10px; }

table { width: 100%; border-collapse: collapse; margin-top: 10px; }

table, th, td { border: 1px solid #e9ecef; }

```

```
th, td { padding: 10px; text-align: left; }
th { background-color: #f8f9fa; }
```

3. Berkas app.js (Logika Komunikasi & UI Dinamis Berdasarkan Role)

JavaScript

```
const URL_API = 'http://127.0.0.1:8000/api';

const kotakLogin = document.getElementById('kotak-login');
const kotakDashboard = document.getElementById('kotak-dashboard');
const formLogin = document.getElementById('form-login');
const pesanError = document.getElementById('pesan-error');
const btnLogout = document.getElementById('btn-logout');
const labelUser = document.getElementById('label-user');
const badgeRole = document.getElementById('badge-role');
const areaTambahNilai = document.getElementById('area-tambah-nilai');
const formNilai = document.getElementById('form-nilai');
const tabelData = document.getElementById('tabel-data');

document.addEventListener('DOMContentLoaded', () => {
  let token = localStorage.getItem('token_rbac');
  if (token) {
    muatProfilDanData();
  }
});

// 1. PROSES LOGIN
formLogin.addEventListener('submit', async (e) => {
  e.preventDefault();
  let emailInput = document.getElementById('pilih-role').value;
  let passwordInput = document.getElementById('password').value;

  try {
    let response = await fetch(`${URL_API}/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
      body: JSON.stringify({ email: emailInput, password: passwordInput })
    });

    let result = await response.json();

    if (response.ok && result.status) {
      localStorage.setItem('token_rbac', result.access_token);
    }
  }
});
```

```

        muatProfilDanData();
    } else {
        pesanError.textContent = result.message || "Login gagal!";
    }
} catch (err) {
    pesanError.textContent = "Gagal terhubung ke server backend Laravel!";
}
});

```

// 2. AMBIL DATA USER (CEK ROLE UNTUK MENGATUR TAMPILAN FRONTEND)

```

async function muatProfilDanData() {
    let token = localStorage.getItem('token_rbac');

    try {
        let response = await fetch(`${URL_API}/me`, {
            headers: { 'Authorization': `Bearer ${token}`, 'Accept': 'application/json' }
        });
    }

```

```

    if (!response.ok) {
        logout();
        return;
    }

```

```

    let result = await response.json();
    let user = result.user;

```

```

    // Tampilkan Identitas Profil
    labelUser.textContent = user.name;
    badgeRole.textContent = user.role;
    badgeRole.className = `badge ${user.role}`;

```

// SESUAIKAN TAMPILAN SESUAI HAK AKSES ROLE (UI ADAPTIVE)

```

    if (user.role === 'admin' || user.role === 'guru') {
        areaTambahNilai.classList.remove('sembunyi');
    } else {
        areaTambahNilai.classList.add('sembunyi');
    }

```

```

    kotakLogin.classList.add('sembunyi');
    kotakDashboard.classList.remove('sembunyi');

```

```

    // Muat Data Nilai
    ambilDataNilai(user.role);

```

```

    } catch (err) {
      console.error("Error profil:", err);
    }
  }
}

```

// 3. GET DATA NILAI

```

async function ambilDataNilai(userRole) {
  let token = localStorage.getItem('token_rbac');

  try {
    let response = await fetch(`${URL_API}/grades`, {
      headers: { 'Authorization': `Bearer ${token}`, 'Accept': 'application/json' }
    });

    let result = await response.json();

    if (response.ok) {
      let html = "";
      result.data.forEach(item => {
        html += `
          <tr>
            <td>${item.id}</td>
            <td>${item.student ? item.student.name : 'Siswa ID: ' + item.user_id}</td>
            <td>${item.subject}</td>
            <td><strong>${item.score}</strong></td>
            <td>
              ${userRole === 'admin'
                ? `<button onclick="hapusNilai(${item.id})" class="btn merah" style="padding: 3px 8px; font-size: 11px;">Hapus (Admin Only)</button>`
                : `<span style="color:#aaa; font-size:12px;">No Access</span>`}
            </td>
          </tr>
        `;
      });
      tabelData.innerHTML = html;
    }
  } catch (err) {
    console.error("Error nilai:", err);
  }
}

```

// 4. POST SIMPAN NILAI (ADMIN & GURU)

```

formNilai.addEventListener('submit', async (e) => {
  e.preventDefault();

```

```

let token = localStorage.getItem('token_rbac');

let payload = {
  user_id: document.getElementById('input-siswa-id').value,
  subject: document.getElementById('input-mapel').value,
  score: document.getElementById('input-nilai').value
};

try {
  let response = await fetch(`${URL_API}/grades`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`,
      'Accept': 'application/json'
    },
    body: JSON.stringify(payload)
  });

  let result = await response.json();

  if (response.ok) {
    alert('Nilai berhasil disimpan!');
    formNilai.reset();
    muatProfilDanData();
  } else {
    alert('Gagal Simpan (HTTP ' + response.status + '): ' + result.message);
  }
} catch (err) {
  alert('Terjadi kesalahan jaringan/server:');
}
});

```

// 5. DELETE NILAI (EKSKLUSIF ADMIN)

```

async function hapusNilai(id) {
  if (!confirm('Apakah Anda yakin ingin menghapus data nilai ini?')) return;

```

```

  let token = localStorage.getItem('token_rbac');

  try {
    let response = await fetch(`${URL_API}/grades/${id}`, {
      method: 'DELETE',
      headers: {

```

```
    'Authorization': `Bearer ${token}`,  
    'Accept': 'application/json'  
  }  
});
```

```
let result = await response.json();
```

```
  if (response.ok) {  
    alert(result.message);  
    muatProfilDanData();  
  } else {  
    alert('Akses Ditolak (HTTP ' + response.status + '): ' + result.message);  
  }  
} catch (err) {  
  alert('Gagal menghapus data.');
```

```
// 6. LOGOUT
```

```
function logout() {  
  localStorage.removeItem('token_rbac');  
  kotakLogin.classList.remove('sembunyi');  
  kotakDashboard.classList.add('sembunyi');  
}
```

```
btnLogout.addEventListener('click', logout);
```

BAGIAN 3: PANDUAN SIMULASI PENGUJIAN RBAC (E2E TESTING)

Lakukan pengujian skenario otorisasi berikut bersama peserta didik di laboratorium komputer:

Skenario Pengujian	Akun Dites (Role)	Aksi yang Dilakukan	Hasil yang Diharapkan	Keterangan Verifikasi Keamanan
Skenario 1	Siswa	Login ke sistem.	Form Tambah Nilai otomatis Sembunyi di UI.	Pengujian Responsif UI.
Skenario 2	Siswa	Tembak endpoint POST /api/grades via Console / Postman.	HTTP 403 Forbidden.	Custom Middleware CheckRole bekerja memblokir akses di level route.
Skenario 3	Guru	Login dan tambah nilai baru.	Data Berhasil Disimpan.	Guru diizinkan oleh Middleware role:admin,guru.
Skenario 4	Guru	Coba klik tombol/panggil rute DELETE /api/grades/{id}.	HTTP 403 Forbidden.	Middleware role:admin memblokir Guru.
Skenario 5	Admin	Login dan hapus data nilai.	Data Berhasil Dihapus.	Admin memiliki hak penuh (Full Access).